

CI/CD Pipeline for a Dockerized FastAPI App on AWS

Sep 30, 2026 · @Sheharzad

30-minute fast track (use this in the timed task)

In 30 minutes, skip the hand-built load balancer and use **Amazon ECS Express Mode**: you give it an image from ECR and it creates the Fargate service, load balancer, security groups, logs and a public HTTPS URL for you. GitHub Actions does test → build → push to ECR. Steps 5–7 below remain the "full version" to talk about if they ask how you'd do it in production.

Minutes	Do this
0–6	Create the repo and add the code files (ideally prepared before the timer starts)
6–9	Create the ECR repository
9–13	Create an IAM user with an access key; add it to GitHub secrets
13–17	Push to <code>main</code> ; watch Actions go green; image appears in ECR
17–27	Create the ECS Express Mode service from that image; wait for it to become active
27–30	Open the URL, test <code>/health</code> and <code>/docs</code> , finish the README

A. Code (0–6 min)

Copy these files exactly from Steps 1–3: `app/__init__.py` (empty), `app/models.py`, `app/storage.py`, `app/main.py`, `tests/__init__.py` (empty), `tests/test_api.py`, `requirements.txt`, `requirements-dev.txt`, `pyproject.toml`, `Dockerfile`, `.dockerignore`, `.gitignore`. Keep them in a GitHub Gist or a private repo before the test so you only paste and push. If you can, run `pytest` once locally.

B. ECR repository (6–9 min)

1. Console top-right: region **us-east-1**.

2. Search **ECR** → **Create repository** → name `task-api` → tag mutability **Mutable** (so we can also push a `latest` tag) → **Create**.

C. AWS access for GitHub (9-13 min)

For speed, use an access key instead of OIDC (say in the write-up that production should use OIDC, as in Step 5).

1. **IAM** → **Users** → **Create user** → name `github-actions-task-api` → Next.
2. **Attach policies directly** → tick `AmazonEC2ContainerRegistryPowerUser` → Next → **Create user**.
3. Open the user → **Permissions** → **Add permissions** → **Create inline policy** → **JSON**, paste this, name it `ecs-redeploy`, create:

```
{
  "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow", "Action": ["ecs:UpdateService",
"ecs:DescribeServices"], "Resource": "*" }]
```

4. Open the user → **Security credentials** → **Create access key** → use case **Third-party service** (or Other) → Create → copy both values (you can't see the secret again).
5. GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**: `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.

D. Workflow `.github/workflows/ci-cd.yml` (paste, commit, push)

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: task-api
  ECS_CLUSTER: default
  ECS_SERVICE: task-api

jobs:
  test:
```

```

name: Lint and test
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: "3.12"
      cache: pip
  - run: pip install -r requirements-dev.txt
  - run: ruff check .
  - run: pytest

build-push-deploy:
name: Build, push, deploy
needs: test
if: github.event_name != 'pull_request'
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4

  - uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: ${ env.AWS_REGION }

  - id: ecr
    uses: aws-actions/amazon-ecr-login@v2

  - name: Build and push image
    env:
      REGISTRY: ${ steps.ecr.outputs.registry }
    run: |
      IMAGE=$REGISTRY/$ECR_REPOSITORY
      docker build --build-arg APP_VERSION=${ github.sha } \
        -t $IMAGE:${ github.sha } -t $IMAGE:latest .
      docker push $IMAGE:${ github.sha }
      docker push $IMAGE:latest

  - name: Redeploy ECS service (skips until the service exists)
    run: |
      if aws ecs describe-services --cluster $ECS_CLUSTER --services
$ECS_SERVICE \
        --query 'services[?status=='ACTIVE']' --output text | grep -q
.; then
      aws ecs update-service --cluster $ECS_CLUSTER --service

```

```
$ECS_SERVICE --force-new-deployment
else
    echo "Service not created yet - skipping deploy"
fi
```

The redeploy step works because the service runs the `latest` tag: forcing a new deployment makes ECS start fresh tasks that pull the newest image.

E. ECS Express Mode service (17–27 min)

1. Search **ECS** → left menu **Express mode**.
2. **Image URI**: browse ECR → repository `task-api` → tag `latest`.
3. **Task execution role** and **Infrastructure role**: choose **Create new role** for each (the console creates `ecsTaskExecutionRole` and the Express infrastructure role).
4. Open **Additional configurations**:
 - Cluster: **default** (created for you if missing); Service name: `task-api`
 - **Container port**: **8000** (the default is 80, which would fail)
 - **Health check path**: `/health` (the default is `/ping`, which our API doesn't have)
 - CPU 0.25 vCPU, memory 0.5 GB (defaults are fine)
5. **Create**. Watch the resources come up; it takes several minutes while the load balancer is provisioned.
6. When the service is active, copy the **Application URL** shown on the service page.

F. Verify and show (27–30 min)

- Browser: `<Application URL>/health` → `{"status":"ok","version":"<commit sha>"}`; `<Application URL>/docs` → create a task.
- GitHub Actions tab: green run; ECR: image tagged with the SHA.
- Prove CD: change the `description=` text in `app/main.py`, push, wait for the run, refresh `/health` — the `version` changes to the new SHA.

If something goes wrong: container port or health-check path is the usual culprit (tasks start then stop). Check ECS → service → **Events** and **Logs** tabs.

README in 3 minutes: paste the architecture diagram from 7.1, the run steps (local: `docker compose up`; deploy: push to `main`), and three troubleshooting rows (wrong port, wrong health path, AccessDenied on push).

Overview

You will build a Task API in FastAPI, test it with pytest, package it with Docker, and ship it through GitHub Actions to Amazon ECR and Amazon ECS on Fargate. Every push to `main` runs tests, builds the image, pushes it, and rolls out a new version automatically.

Deployment target assumption: the brief does not say which AWS service the CDIS department uses, so this guide deploys to **ECS Fargate behind an Application Load Balancer** (serverless containers, no servers to patch, the common choice for department-level APIs). Step 6 lists App Runner and EC2 as alternatives if CDIS prefers those.

Layer	Tool	Why
API	FastAPI + Pydantic v2	Fast, typed, auto-generated OpenAPI docs at <code>/docs</code>
Tests	pytest + FastAPI TestClient (httpx)	Tests hit real HTTP routes without a running server
Lint	Ruff	One fast tool for linting and import order
Container	Docker (multi-stage, non-root)	Same image runs locally, in CI, and on AWS
CI/CD	GitHub Actions	Built into GitHub, free minutes for public repos
Registry	Amazon ECR	Private registry in the same account as ECS
Runtime	Amazon ECS on Fargate + ALB	Managed containers, health checks, rolling deploys
Logs	Amazon CloudWatch Logs	Container stdout lands here automatically
Auth to AWS	GitHub OIDC + IAM role	No long-lived AWS keys stored in GitHub

Prerequisites checklist

- ☐ Python 3.12 installed (`python --version`)
- ☐ Docker Desktop (or Docker Engine) running (`docker --version`)
- ☐ Git + a GitHub account and an empty repo, e.g. `task-api`
- ☐ AWS account with admin access (or a user that can manage ECR, ECS, IAM, EC2, load balancers and CloudWatch)
- ☐ Able to sign in to the AWS Management Console (`console.aws.amazon.com`); no AWS CLI needed

- A region picked in the console's top-right corner; this guide uses `us-east-1`, change it everywhere if CDIS uses another

Final repository layout

```
task-api/
├── app/
│   ├── __init__.py
│   ├── main.py          # FastAPI app, routes, error handlers
│   ├── models.py        # Pydantic schemas (validation)
│   └── storage.py        # In-memory task repository
├── tests/
│   ├── __init__.py
│   └── test_api.py       # Automated API tests
├── .aws/
│   └── task-definition.json
├── .github/
│   └── workflows/
│       └── ci-cd.yml      # Test → build → push → deploy
├── .dockerignore
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml        # pytest + ruff config
├── requirements.txt
├── requirements-dev.txt
└── README.md
```

Step 1 — Build the FastAPI application

The API exposes full CRUD on tasks plus `/health`. Storage is an in-memory dictionary behind a small repository class, so you can swap in DynamoDB or RDS later without touching the routes.

Method	Path	Purpose	Success code
GET	<code>/health</code>	Liveness check for Docker, ALB and ECS	200
GET	<code>/tasks</code>	List tasks, optional <code>?completed=true/false</code> filter	200
POST	<code>/tasks</code>	Create a task	201
GET	<code>/tasks/{task_id}</code>	Get one task	200

Method	Path	Purpose	Success code
PUT	/tasks/{task_id}	Replace/update a task (partial fields allowed)	200
DELETE	/tasks/{task_id}	Delete a task	204

1.1 Create the project

```
mkdir task-api && cd task-api
git init -b main
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
mkdir app tests
touch app/__init__.py tests/__init__.py
```

1.2 requirements.txt (runtime) and requirements-dev.txt (tests + lint)

```
# requirements.txt
fastapi==0.115.0
uvicorn[standard]==0.30.6
pydantic==2.9.2
```

```
# requirements-dev.txt
-r requirements.txt
pytest==8.3.3
httpx==0.27.2
ruff==0.6.9
```

```
pip install -r requirements-dev.txt
```

Pinned versions keep CI builds reproducible. Bump them deliberately, not by accident.

1.3 app/storage.py — in-memory repository

```
from datetime import datetime, timezone
from threading import Lock
from uuid import UUID, uuid4

from app.models import Task, TaskCreate, TaskUpdate

class TaskNotFoundError(Exception):
```

```

def __init__(self, task_id: UUID):
    self.task_id = task_id
    super().__init__(f"Task {task_id} not found")

class TaskRepository:
    """Thread-safe in-memory store. Replace with DynamoDB/RDS later."""

    def __init__(self) -> None:
        self._tasks: dict[UUID, Task] = {}
        self._lock = Lock()

    def list(self, completed: bool | None = None) -> list[Task]:
        tasks = list(self._tasks.values())
        if completed is not None:
            tasks = [t for t in tasks if t.completed == completed]
        return sorted(tasks, key=lambda t: t.created_at)

    def get(self, task_id: UUID) -> Task:
        task = self._tasks.get(task_id)
        if task is None:
            raise TaskNotFoundError(task_id)
        return task

    def create(self, data: TaskCreate) -> Task:
        now = datetime.now(timezone.utc)
        task = Task(id=uuid4(), created_at=now, updated_at=now,
**data.model_dump())
        with self._lock:
            self._tasks[task.id] = task
        return task

    def update(self, task_id: UUID, data: TaskUpdate) -> Task:
        with self._lock:
            current = self.get(task_id)
            changes = data.model_dump(exclude_unset=True)
            updated = current.model_copy(
                update={**changes, "updated_at": datetime.now(timezone.utc)}
            )
            self._tasks[task_id] = updated
        return updated

    def delete(self, task_id: UUID) -> None:
        with self._lock:
            self.get(task_id)
            del self._tasks[task_id]

```

```
def clear(self) -> None:
    with self._lock:
        self._tasks.clear()
```

1.4 app/main.py — routes and health check

```
import logging
import os
from uuid import UUID

from fastapi import FastAPI, Request, status
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

from app.models import HealthResponse, Task, TaskCreate, TaskUpdate
from app.storage import TaskNotFoundError, TaskRepository

APP_VERSION = os.getenv("APP_VERSION", "dev")

logging.basicConfig(
    level=os.getenv("LOG_LEVEL", "INFO"),
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
)
logger = logging.getLogger("task-api")

app = FastAPI(
    title="Task Management API",
    version=APP_VERSION,
    description="Simple task API deployed via GitHub Actions to AWS ECS Fargate.",
)

repo = TaskRepository()

# ----- Error handlers (Step 2) -----
@app.exception_handler(TaskNotFoundError)
async def task_not_found_handler(request: Request, exc: TaskNotFoundError):
    return JSONResponse(
        status_code=status.HTTP_404_NOT_FOUND,
        content={"error": "not_found", "detail": str(exc)},
    )

@app.exception_handler(RequestValidationError)
```

```

async def validation_handler(request: Request, exc: RequestValidationError):
    errors = [
        {"field": ".".join(str(p) for p in e["loc"][1:]), "message":
e["msg"]}
        for e in exc.errors()
    ]
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content={"error": "validation_error", "detail": errors},
    )

@app.exception_handler(Exception)
async def unhandled_handler(request: Request, exc: Exception):
    logger.exception("Unhandled error on %s %s", request.method,
request.url.path)
    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={"error": "internal_error", "detail": "Something went
wrong"},
    )

# ----- Routes -----
@app.get("/health", response_model=HealthResponse, tags=["health"])
def health() -> HealthResponse:
    return HealthResponse(status="ok", version=APP_VERSION)

@app.get("/tasks", response_model=list[Task], tags=["tasks"])
def list_tasks(completed: bool | None = None) -> list[Task]: # query param
    return repo.list(completed=completed)

@app.post("/tasks", response_model=Task, status_code=status.HTTP_201_CREATED,
tags=["tasks"])
def create_task(payload: TaskCreate) -> Task:
    task = repo.create(payload)
    logger.info("Created task %s", task.id)
    return task

@app.get("/tasks/{task_id}", response_model=Task, tags=["tasks"])
def get_task(task_id: UUID) -> Task:
    return repo.get(task_id)

```

```

@app.put("/tasks/{task_id}", response_model=Task, tags=["tasks"])
def update_task(task_id: UUID, payload: TaskUpdate) -> Task:
    return repo.update(task_id, payload)

@app.delete("/tasks/{task_id}", status_code=status.HTTP_204_NO_CONTENT, tags=
["tasks"])
def delete_task(task_id: UUID) -> None:
    repo.delete(task_id)

```

1.5 Run it locally (without Docker)

```
uvicorn app.main:app --reload --port 8000
```

Open <http://localhost:8000/docs> for Swagger UI, then try:

```

curl http://localhost:8000/health
curl -X POST http://localhost:8000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Write CI pipeline", "priority": "high"}'
curl http://localhost:8000/tasks

```

`app/models.py` is in Step 2; create it before running.

Step 2 — Validation, error handling and automated tests

Pydantic rejects bad input before it reaches your code, the handlers in `main.py` turn every failure into a consistent JSON shape, and pytest proves both.

Situation	Status	Response body
Missing/blank title, title > 200 chars, bad priority	422	<code>{"error": "validation_error", "detail": [{"field", "message"}]}</code>
Malformed UUID in the path	422	same validation shape
Update with an empty body <code>{}</code>	422	same validation shape
Task id does not exist	404	<code>{"error": "not_found", "detail": "Task ... not found"}</code>
Any unexpected exception	500	<code>{"error": "internal_error", ...}</code> and a full stack trace in the logs

2.1 app/models.py — schemas and rules

```

from datetime import date, datetime
from enum import Enum
from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field, field_validator,
model_validator

class Priority(str, Enum):
    low = "low"
    medium = "medium"
    high = "high"

class TaskBase(BaseModel):
    model_config = ConfigDict(extra="forbid") # reject unknown fields

    title: str = Field(..., min_length=1, max_length=200)
    description: str | None = Field(default=None, max_length=1000)
    priority: Priority = Priority.medium
    due_date: date | None = None
    completed: bool = False

    @field_validator("title")
    @classmethod
    def title_not_blank(cls, v: str) -> str:
        v = v.strip()
        if not v:
            raise ValueError("title must not be blank")
        return v

class TaskCreate(TaskBase):
    pass

class TaskUpdate(BaseModel):
    model_config = ConfigDict(extra="forbid")

    title: str | None = Field(default=None, min_length=1, max_length=200)
    description: str | None = Field(default=None, max_length=1000)
    priority: Priority | None = None
    due_date: date | None = None
    completed: bool | None = None

```

```

@field_validator("title")
@classmethod
def title_not_blank(cls, v: str | None) -> str | None:
    if v is not None and not v.strip():
        raise ValueError("title must not be blank")
    return v.strip() if v else v

@model_validator(mode="after")
def at_least_one_field(self):
    if not self.model_fields_set:
        raise ValueError("provide at least one field to update")
    return self

class Task(TaskBase):
    id: UUID
    created_at: datetime
    updated_at: datetime

class HealthResponse(BaseModel):
    status: str
    version: str

```

2.2 pyproject.toml — pytest and Ruff config

```

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v"

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP"]

```

2.3 tests/test_api.py — automated API tests

```

import uuid

import pytest
from fastapi.testclient import TestClient

```

```

from app.main import app, repo

client = TestClient(app)

@pytest.fixture(autouse=True)
def clean_repo():
    repo.clear()
    yield
    repo.clear()

def make_task(**overrides):
    body = {"title": "Write tests", "priority": "high"} | overrides
    resp = client.post("/tasks", json=body)
    assert resp.status_code == 201, resp.text
    return resp.json()

# ----- Health -----
def test_health_returns_ok():
    resp = client.get("/health")
    assert resp.status_code == 200
    assert resp.json()["status"] == "ok"

# ----- Create -----
def test_create_task_returns_201_and_defaults():
    task = make_task(title=" Deploy to AWS ")
    assert task["title"] == "Deploy to AWS" # whitespace trimmed
    assert task["completed"] is False
    assert uuid.UUID(task["id"])

@pytest.mark.parametrize(
    "body",
    [
        {}, # missing title
        {"title": ""}, # empty
        {"title": " "}, # blank
        {"title": "x" * 201}, # too long
        {"title": "ok", "priority": "urgent"}, # bad enum
        {"title": "ok", "due_date": "not-a-date"},
        {"title": "ok", "unknown": 1}, # extra field
    ],
)

```

```

def test_create_task_rejects_invalid_input(body):
    resp = client.post("/tasks", json=body)
    assert resp.status_code == 422
    assert resp.json()["error"] == "validation_error"

# ----- Read -----
def test_list_and_filter_tasks():
    make_task(title="A")
    make_task(title="B", completed=True)
    assert len(client.get("/tasks").json()) == 2
    done = client.get("/tasks", params={"completed": True}).json()
    assert [t["title"] for t in done] == ["B"]

def test_get_task_by_id():
    task = make_task()
    resp = client.get(f"/tasks/{task['id']}")
    assert resp.status_code == 200
    assert resp.json()["id"] == task["id"]

def test_get_missing_task_returns_404():
    resp = client.get(f"/tasks/{uuid.uuid4()}")
    assert resp.status_code == 404
    assert resp.json()["error"] == "not_found"

def test_get_task_with_bad_uuid_returns_422():
    assert client.get("/tasks/not-a-uuid").status_code == 422

# ----- Update -----
def test_update_task_partial():
    task = make_task()
    resp = client.put(f"/tasks/{task['id']}", json={"completed": True})
    assert resp.status_code == 200
    body = resp.json()
    assert body["completed"] is True
    assert body["title"] == task["title"]
    assert body["updated_at"] >= task["updated_at"]

def test_update_with_empty_body_returns_422():
    task = make_task()
    assert client.put(f"/tasks/{task['id']}", json={}).status_code == 422

```

```
def test_update_missing_task_returns_404():
    resp = client.put(f"/tasks/{uuid.uuid4()}", json={"completed": True})
    assert resp.status_code == 404

# ----- Delete -----
def test_delete_task():
    task = make_task()
    assert client.delete(f"/tasks/{task['id']}").status_code == 204
    assert client.get(f"/tasks/{task['id']}").status_code == 404

def test_delete_missing_task_returns_404():
    assert client.delete(f"/tasks/{uuid.uuid4()}").status_code == 404
```

2.4 Run the tests and linter

```
ruff check .
pytest
```

Expected: `ruff` prints `All checks passed!` and `pytest` reports **18 passed** (12 test functions, one of which runs 7 invalid-input cases). Add `pytest --cov=app` with `pytest-cov` if the reviewer asks for coverage numbers.

2.5 Commit

```
cat > .gitignore <<'EOF'
.venv/
__pycache__/
*.pyc
.pytest_cache/
.ruff_cache/
.env
EOF
git add .
git commit -m "feat: task API with validation, error handling and tests"
```

Step 3 — Containerize with Docker and run locally

A two-stage build installs dependencies in a builder image and copies only the result into a slim runtime image that runs as a non-root user. The final image is roughly 150–200 MB instead of 1 GB+.

3.1 Dockerfile

```
# ----- Stage 1: build dependencies -----
FROM python:3.12-slim AS builder

ENV PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build
COPY requirements.txt .
RUN python -m venv /opt/venv \
    && /opt/venv/bin/pip install -r requirements.txt

# ----- Stage 2: runtime -----
FROM python:3.12-slim AS runtime

ARG APP_VERSION=dev
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_VERSION=${APP_VERSION} \
    PORT=8000

# Non-root user
RUN groupadd --system app && useradd --system --gid app --no-create-home app

WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY app/ ./app/

USER app
EXPOSE 8000

# Container-level health check (slim image has no curl, so use Python)
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request,sys; sys.exit(0 if
```

```
urllib.request.urlopen('http://127.0.0.1:8000/health').status==200 else 1)"

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--
proxy-headers"]
```

Why each choice matters, in case the reviewer asks:

- `--host 0.0.0.0` — without it uvicorn listens only inside the container and the port mapping shows nothing.
- `PYTHONUNBUFFERED=1` — logs reach `docker logs` and CloudWatch immediately instead of sitting in a buffer.
- `USER app` — a compromised process cannot write to system paths.
- `--proxy-headers` — correct client IP and scheme when running behind the ALB.
- Copying `requirements.txt` before `app/` — the dependency layer stays cached when only code changes.

3.2 `.dockerignore`

```
.git
.github
.venv
__pycache__
*.pyc
.pytest_cache
.ruff_cache
tests
.aws
.env
README.md
docker-compose.yml
```

3.3 docker-compose.yml (local convenience)

```
services:
  api:
    build:
      context: .
      args:
        APP_VERSION: local
    image: task-api:local
    ports:
      - "8000:8000"
    environment:
      LOG_LEVEL: DEBUG
    restart: unless-stopped
```

3.4 Build, run and verify

```
# Option A: plain Docker
docker build -t task-api:local --build-arg APP_VERSION=local .
docker run -d --name task-api -p 8000:8000 task-api:local

# Option B: Compose
docker compose up --build -d

# Verify
docker ps                                # STATUS should become "
(healthy)"                               #
curl http://localhost:8000/health         #
{"status":"ok","version":"local"}
curl -X POST http://localhost:8000/tasks -H "Content-Type: application/json" \
  -d '{"title":"From Docker"}'
docker logs -f task-api

# Inspect size and user
docker images task-api
docker exec task-api whoami              # app

# Clean up
docker rm -f task-api    # or: docker compose down
```

3.5 Optional: run tests inside the image build context

To prove the container's Python matches CI, run the suite in a throwaway container:

```
docker run --rm -v "$PWD":/src -w /src python:3.12-slim \
  sh -c "pip install -q -r requirements-dev.txt && pytest -q"
```

Commit: `git add . && git commit -m "build: add Dockerfile and compose"`.

Step 4 — GitHub Actions: run tests and build the image

Start with CI only: every pull request and every push to `main` lints, tests, then builds the Docker image and smoke-tests `/health` inside it. Nothing is pushed yet; Steps 5 and 6 extend this same file.

4.1 Push the repo to GitHub

```
git remote add origin https://github.com/<your-user>/task-api.git
git push -u origin main
```

4.2 `.github/workflows/ci-cd.yml` (CI version)

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    name: Lint and test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-dev.txt
```

- name: Install dependencies
run: pip install -r requirements-dev.txt
- name: Lint (ruff)
run: ruff check .
- name: Run tests
run: pytest --junitxml=test-results.xml
- name: Upload test report
if: always()
uses: actions/upload-artifact@v4
with:
 name: test-results
 path: test-results.xml

build:

- name: Build Docker image
- needs: test
- runs-on: ubuntu-latest
- steps:
 - uses: actions/checkout@v4
 - uses: docker/setup-buildx-action@v3
 - name: Build image (load locally, no push)
uses: docker/build-push-action@v6
with:
 context: .
 load: true
 push: false
 tags: task-api:ci
 build-args: APP_VERSION=\${{ github.sha }}
 cache-from: type=gha
 cache-to: type=gha,mode=max
 - name: Smoke test the container
run: |
 docker run -d --name smoke -p 8000:8000 task-api:ci
 for i in \$(seq 1 15); do
 if curl -fsS http://localhost:8000/health; then echo; exit 0; fi
 sleep 2
 done
 echo "Container never became healthy"; docker logs smoke; exit 1

4.3 What each part does

- `needs: test` — the image is built only when lint and tests pass, so a broken commit never becomes an image.
- `concurrency` — a newer push to the same branch cancels the older run, saving minutes.
- `cache-from/cache-to: type=gha` — Docker layers are cached in GitHub's cache; a code-only change rebuilds in seconds.
- Smoke test — proves the built image actually starts and serves `/health`, not just that it compiled.

4.4 Protect `main` (recommended)

GitHub → repo **Settings** → **Branches** → **Add branch ruleset** for `main`: require a pull request and require the **Lint and test** and **Build Docker image** checks to pass. Now nothing untested can merge.

4.5 Verify

Commit and push the workflow, then open the **Actions** tab. Both jobs should be green. To see the gate work, push a branch that breaks a test and open a PR: the `build` job is skipped and the PR shows a red check.

Step 5 — Push the image to Amazon ECR

GitHub Actions gets short-lived AWS credentials through OpenID Connect (OIDC), assumes an IAM role that only your repo's `main` branch can use, and pushes the image to ECR tagged with the commit SHA. No AWS access keys are ever stored in GitHub.

Everything in Steps 5–6 is done in the **AWS Management Console** in your browser (<https://console.aws.amazon.com>). Use the search bar at the top to jump to each service. Menu labels can shift slightly as AWS updates the console; the field names below are what to look for.

Before you start: pick your region in the top-right corner (this guide uses **US East (N. Virginia) us-east-1**) and keep it the same for every step. Click your account name in the top-right and copy the 12-digit **Account ID**; you'll paste it into two JSON documents below as `<ACCOUNT_ID>`.

5.1 Create the ECR repository

1. Search **ECR** → open **Elastic Container Registry**.
2. Left menu **Private registry** → **Repositories** → **Create repository**.

3. Repository name: `task-api`.
4. Image tag mutability: **Immutable** (a commit-SHA tag can never be overwritten, so every deploy traces to one commit).
5. Leave encryption as AES-256 → **Create**.
6. Turn on scan on push: left menu **Private registry** → **Features & Settings** → **Scanning** → **Basic scanning** → add a filter `*` with **Scan on push** ticked → Save.
7. Open the `task-api` repository → left menu **Lifecycle policy** → **Create rule**:
 - Rule priority `1`, description `Keep last 20 images`
 - Image status **Any**, match criteria **Image count more than** `20` → Save.
8. Copy the repository **URI** (looks like `123456789012.dkr.ecr.us-east-1.amazonaws.com/task-api`). You'll need it in Step 6.

5.2 Register GitHub as an identity provider (once per AWS account)

1. Search **IAM** → left menu **Identity providers** → **Add provider**.
2. Provider type: **OpenID Connect**.
3. Provider URL: `https://token.actions.githubusercontent.com`
4. Audience: `sts.amazonaws.com`
5. **Add provider**. If it says the provider already exists, skip this step.

5.3 Create the deploy role for GitHub Actions

Create the role:

1. IAM → **Roles** → **Create role**.
2. Trusted entity type: **Web identity**.
3. Identity provider: `token.actions.githubusercontent.com`; Audience: `sts.amazonaws.com`.
4. GitHub organization: your GitHub username (e.g. `sheharzad-developer`); GitHub repository: `task-api`; GitHub branch: `main` → **Next**.
5. Skip the permissions page → **Next**.
6. Role name: `github-actions-task-api` → **Create role**.

Allow the production environment too. The deploy job uses `environment: production`, which presents a different identity to AWS, so the trust policy must list both. Open the role → **Trust relationships** tab → **Edit trust policy** → replace everything with this (swap in your Account ID and GitHub username) → **Update policy**:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-
provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": { "token.actions.githubusercontent.com:aud":
"sts.amazonaws.com" },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": [
          "repo:<GITHUB_OWNER>/task-api:ref:refs/heads/main",
          "repo:<GITHUB_OWNER>/task-api:environment:production"
        ]
      }
    }
  }]
}
```

Add the permissions. Same role → **Permissions** tab → **Add permissions** → **Create inline policy** → switch the editor to **JSON** → paste this (swap in your Account ID) → **Next** → Policy name `task-api-deploy` → **Create policy**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "EcrAuth", "Effect": "Allow", "Action":
"ecr:GetAuthorizationToken", "Resource": "*" },
    {
      "Sid": "EcrPush", "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability", "ecr:InitiateLayerUpload",
"ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload", "ecr:PutImage", "ecr:BatchGetImage",
"ecr:DescribeImages"
      ],
      "Resource": "arn:aws:ecr:us-east-1:<ACCOUNT_ID>:repository/task-api"
    },
    {
      "Sid": "EcsDeploy", "Effect": "Allow",
      "Action": ["ecs:RegisterTaskDefinition", "ecs:DescribeTaskDefinition",
"ecs:UpdateService", "ecs:DescribeServices"],
```

```

    "Resource": "*"
  },
  {
    "Sid": "PassExecutionRole", "Effect": "Allow", "Action":
"iam:PassRole",
    "Resource": "arn:aws:iam::<ACCOUNT_ID>:role/ecsTaskExecutionRole"
  }
]
}

```

Finally, copy the role's **ARN** from the top of its Summary page (e.g. `arn:aws:iam::123456789012:role/github-actions-task-api`) for the GitHub variables below.

5.4 Add GitHub repository variables

Repo → **Settings** → **Secrets and variables** → **Actions** → **Variables** (these are not secret, so use Variables, not Secrets):

Name	Example value
<code>AWS_REGION</code>	<code>us-east-1</code>
<code>AWS_ROLE_ARN</code>	<code>arn:aws:iam::123456789012:role/github-actions-task-api</code>
<code>ECR_REPOSITORY</code>	<code>task-api</code>
<code>ECS_CLUSTER</code>	<code>task-api-cluster</code> (created in Step 6)
<code>ECS_SERVICE</code>	<code>task-api-service</code> (created in Step 6)
<code>CONTAINER_NAME</code>	<code>task-api</code>

5.5 The push job (added to `ci-cd.yml`)

This job replaces the Step 4 `build` job on pushes to `main`. The full final file is in Step 6.

```

push:
  name: Build and push to ECR
  needs: test
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  permissions:
    id-token: write      # required for OIDC
    contents: read

```

```

outputs:
  image: ${ steps.meta.outputs.image }
steps:
  - uses: actions/checkout@v4

  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: ${ vars.AWS_ROLE_ARN }
      aws-region: ${ vars.AWS_REGION }

  - id: ecr
    uses: aws-actions/amazon-ecr-login@v2

  - id: meta
    run: echo "image=${ steps.ecr.outputs.registry }/${
vars.ECR_REPOSITORY }:${ steps.github.sha }" >> "$GITHUB_OUTPUT"

  - uses: docker/setup-buildx-action@v3

  - uses: docker/build-push-action@v6
    with:
      context: .
      push: true
      tags: ${ steps.meta.outputs.image }
      build-args: APP_VERSION=${ steps.github.sha }
      cache-from: type=gha
      cache-to: type=gha,mode=max

```

5.6 Verify

After the push job runs once (Step 6.5), open **ECR → Repositories → task-api → Images**. You should see an image whose tag is the full commit SHA. Click the tag to see the **Vulnerabilities** scan result; aim for zero Critical findings before the demo.

Step 6 — Deploy to AWS ECS Fargate

You set up the infrastructure once in the AWS Console (6.1–6.7); after that, every merge to `main` deploys itself (6.8). The setup uses the account's **default VPC** to stay simple; a production CDIS setup would use private subnets and a NAT gateway (see 7.4).

6.1 Task execution role (lets ECS pull from ECR and write logs)

1. IAM → **Roles** → **Create role**.

2. Trusted entity type: **AWS service**; Service or use case: **Elastic Container Service** → use case **Elastic Container Service Task** → **Next**.
3. Search and tick **AmazonECSTaskExecutionRolePolicy** → **Next**.
4. Role name: `ecsTaskExecutionRole` (exact spelling; the task definition and the deploy policy both refer to it) → **Create role**.

If a role with this name already exists in the account, reuse it.

6.2 Log group

1. Search **CloudWatch** → left menu **Logs** → **Log groups** → **Create log group**.
2. Name: `/ecs/task-api` ; Retention: **2 weeks** → **Create**.

6.3 Security groups

Two groups: the load balancer accepts the internet on port 80, and the containers accept port 8000 **only from the load balancer**.

1. Search **EC2** → left menu **Network & Security** → **Security Groups** → **Create security group**.
2. Name `task-api-alb-sg` , description `ALB for task-api` , VPC: the **default** VPC.
3. Inbound rules → **Add rule**: Type **HTTP**, Port 80, Source **Anywhere-IPv4** (0.0.0.0/0) → **Create security group**.
4. Create a second group: name `task-api-svc-sg` , description `ECS tasks for task-api` , default VPC.
5. Inbound rule: Type **Custom TCP**, Port `8000` , Source **Custom** → start typing `task-api-alb-sg` and pick it → **Create security group**.

Leave outbound rules as the default (all traffic), so tasks can reach ECR and CloudWatch.

6.4 Target group and Application Load Balancer

Target group first:

1. EC2 → left menu **Load Balancing** → **Target Groups** → **Create target group**.
2. Target type: **IP addresses** (required for Fargate, because each task gets its own network interface).
3. Name `task-api-tg` ; Protocol **HTTP**; Port `8000` ; VPC: default.
4. Health checks: path `/health` . Open **Advanced health check settings**: Healthy threshold `2` , Unhealthy threshold `3` , Interval `15` seconds.
5. **Next** → do not register any IPs (ECS does that automatically) → **Create target group**.

6. Open the new target group → **Attributes** tab → **Edit** → Deregistration delay `30` seconds → Save. This makes deploys finish faster.

Then the load balancer:

1. EC2 → **Load Balancers** → **Create load balancer** → **Application Load Balancer** → Create.
2. Name `task-api-alb`; Scheme **Internet-facing**; IP address type **IPv4**.
3. Network mapping: default VPC, tick **all** Availability Zones/subnets listed.
4. Security groups: remove `default`, select `task-api-alb-sg`.
5. Listeners and routing: **HTTP : 80** → Default action **Forward to** `task-api-tg`.
6. **Create load balancer**. Wait until State is **Active** (2–3 minutes).
7. Copy its **DNS name** (e.g. `task-api-alb-123456.us-east-1.elb.amazonaws.com`). This is your API's public address.

For HTTPS, request a certificate in **AWS Certificate Manager** and add an **HTTPS : 443** listener; mention it in your write-up even if you skip it.

6.5 ECS cluster and the first image

Cluster:

1. Search **ECS** → **Clusters** → **Create cluster**.
2. Cluster name `task-api-cluster`; Infrastructure: **AWS Fargate (serverless)** only.
3. Under Monitoring, turn on **Container Insights** (optional, useful for the demo) → **Create**.

First image: ECS needs an image in ECR before it can start a service. Let the pipeline push it:

1. Make sure the Step 5.4 GitHub variables are set (you can fill `ECS_CLUSTER` and `ECS_SERVICE` now with the names above).
2. Commit the final workflow from 6.8 and push to `main` (or GitHub → **Actions** → **CI/CD** → **Run workflow**).
3. **Lint and test** and **Build and push to ECR** should go green. **Deploy to ECS Fargate** will fail this first time because the service doesn't exist yet. That's expected.
4. In ECR → `task-api` → **Images**, copy the full **Image URI** of the new SHA tag.

6.6 Task definition — `.aws/task-definition.json` (commit this file)

Replace `<ACCOUNT_ID>` and the region. The `image` value is a placeholder; the pipeline swaps in the real SHA-tagged image on every deploy.

```
{
  "family": "task-api",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "runtimePlatform": { "cpuArchitecture": "X86_64", "operatingSystemFamily":
"Linux" },
  "executionRoleArn": "arn:aws:iam::<ACCOUNT_ID>:role/ecsTaskExecutionRole",
  "containerDefinitions": [
    {
      "name": "task-api",
      "image": "<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/task-
api:placeholder",
      "essential": true,
      "portMappings": [{ "containerPort": 8000, "protocol": "tcp" }],
      "environment": [{ "name": "LOG_LEVEL", "value": "INFO" }],
      "healthCheck": {
        "command": ["CMD-SHELL", "python -c \"import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health')\" || exit 1"],
        "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 15
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/task-api",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "api"
        }
      }
    }
  ]
}
```

Register it in the console (first time only):

1. ECS → left menu **Task definitions** → **Create new task definition** → **Create new task definition with JSON**.
2. Delete the sample JSON and paste the file above, with `<ACCOUNT_ID>` filled in and `image` set to the **Image URI** you copied in 6.5.
3. **Create**. You should see `task-api:1`.

After this, the pipeline registers a new revision (`task-api:2` , `:3` ...) on every deploy, using the committed file with the new image swapped in.

6.7 Create the ECS service

1. ECS → **Clusters** → `task-api-cluster` → **Services** tab → **Create**.
2. **Environment / Compute options**: Launch type **FARGATE**, platform version **LATEST**.
3. **Deployment configuration**: Application type **Service**; Family `task-api`, Revision **LATEST**; Service name `task-api-service`; Desired tasks `1`.
4. **Deployment options** (expand): Deployment type **Rolling update**, Min running tasks `100 %`, Max running tasks `200 %`.
5. **Deployment failure detection**: tick **Use the Amazon ECS deployment circuit breaker** and **Rollback on failure**. A release whose tasks keep failing health checks then rolls itself back.
6. **Networking**: VPC default; keep all subnets; Security group **Use an existing security group** → remove `default`, select `task-api-svc-sg`; **Public IP**: turned on (lets tasks in public subnets reach ECR without a NAT gateway).
7. **Load balancing**: Load balancer type **Application Load Balancer** → **Use an existing load balancer** → `task-api-alb`. Container to load balance: `task-api 8000:8000`. Listener **Use an existing listener** → `80:HTTP`. Target group **Use an existing target group** → `task-api-tg`. Health check grace period `30` seconds.
8. **Create**. Open the service and wait until **Deployments** shows Completed and Tasks shows **1/1 Running** (3–5 minutes).
9. Check EC2 → **Target Groups** → `task-api-tg` → **Targets** tab: one target, status **healthy**.
10. In your browser open `http://<ALB DNS name>/health` and then `http://<ALB DNS name>/docs`.

Close the loop: add the GitHub variable `API_URL = http://<ALB DNS name>`, then re-run the failed workflow (Actions → the run → **Re-run failed jobs**). The deploy job should now go green and `/health` should show the commit SHA. From here on, every merge deploys automatically.

6.8 Final `.github/workflows/ci-cd.yml` (test → build → push → deploy)

Replace the Step 4 file with this complete version:

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
```

```

concurrency:
  group: ${GITHUB_WORKFLOW}-${GITHUB_REF}
  cancel-in-progress: false    # never cancel a deploy halfway

permissions:
  contents: read

jobs:
  test:
    name: Lint and test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: pytest --junitxml=test-results.xml
      - if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: test-results.xml

  build:
    name: Build Docker image
    if: github.event_name == 'pull_request'
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v6
        with:
          context: .
          load: true
          tags: task-api:ci
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Smoke test
        run: |
          docker run -d --name smoke -p 8000:8000 task-api:ci
          for i in $(seq 1 15); do curl -fsS localhost:8000/health && exit 0;

```

```

sleep 2; done
    docker logs smoke; exit 1

push:
  name: Build and push to ECR
  if: github.event_name != 'pull_request' && github.ref ==
'refs/heads/main'
  needs: test
  runs-on: ubuntu-latest
  permissions:
    id-token: write
    contents: read
  outputs:
    image: ${ steps.meta.outputs.image }
  steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ vars.AWS_ROLE_ARN }
        aws-region: ${ vars.AWS_REGION }
    - id: ecr
      uses: aws-actions/amazon-ecr-login@v2
    - id: meta
      run: echo "image=${ steps.ecr.outputs.registry }/${
vars.ECR_REPOSITORY }:${ github.sha }" >> "$GITHUB_OUTPUT"
    - uses: docker/setup-buildx-action@v3
    - uses: docker/build-push-action@v6
      with:
        context: .
        platforms: linux/amd64
        push: true
        tags: ${ steps.meta.outputs.image }
        build-args: APP_VERSION=${ github.sha }
        cache-from: type=gha
        cache-to: type=gha,mode=max

deploy:
  name: Deploy to ECS Fargate
  needs: push
  runs-on: ubuntu-latest
  environment: production      # optional: add required reviewers in repo
Settings → Environments
  permissions:
    id-token: write
    contents: read
  steps:

```

```

- uses: actions/checkout@v4
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${ vars.AWS_ROLE_ARN }
    aws-region: ${ vars.AWS_REGION }

- id: render
  uses: aws-actions/amazon-ecs-render-task-definition@v1
  with:
    task-definition: .aws/task-definition.json
    container-name: ${ vars.CONTAINER_NAME }
    image: ${ needs.push.outputs.image }
    environment-variables: |
      APP_VERSION=${ github.sha }

- uses: aws-actions/amazon-ecs-deploy-task-definition@v2
  with:
    task-definition: ${ steps.render.outputs.task-definition }
    cluster: ${ vars.ECS_CLUSTER }
    service: ${ vars.ECS_SERVICE }
    wait-for-service-stability: true

- name: Post-deploy smoke test
  run: |
    for i in $(seq 1 10); do
      body=$(curl -fsS "${ vars.API_URL }/health") && echo "$body" |
grep -q "${ github.sha }" && echo "$body" && exit 0
      sleep 6
    done
    echo "Deployed version not serving"; exit 1

```

The post-deploy check confirms the live `/health` returns the new commit SHA as its `version`, proving the new release, not the old one, is serving traffic.

6.9 If CDIS prefers a different AWS target

Option	Pick it when	Change to this guide
ECS Fargate + ALB (this guide)	Department runs several containerized services, needs VPC control	none

Option	Pick it when	Change to this guide
AWS App Runner	Not available to new accounts: AWS stopped accepting new App Runner customers on April 30, 2026	Use ECS Express Mode instead (see the 30-minute fast track)
EC2 + Docker	Existing EC2 servers, very tight budget	Deploy job SSHes (or uses SSM Run Command) to <code>docker pull</code> and restart the container
Amazon EKS	Department already runs Kubernetes	Replace 6.4–6.8 with a Deployment + Service + Ingress, deploy with <code>kubect1</code> /Helm

Open question to confirm with the department: which AWS account, region and service they standardize on, and whether internal-only (private ALB) access is required.

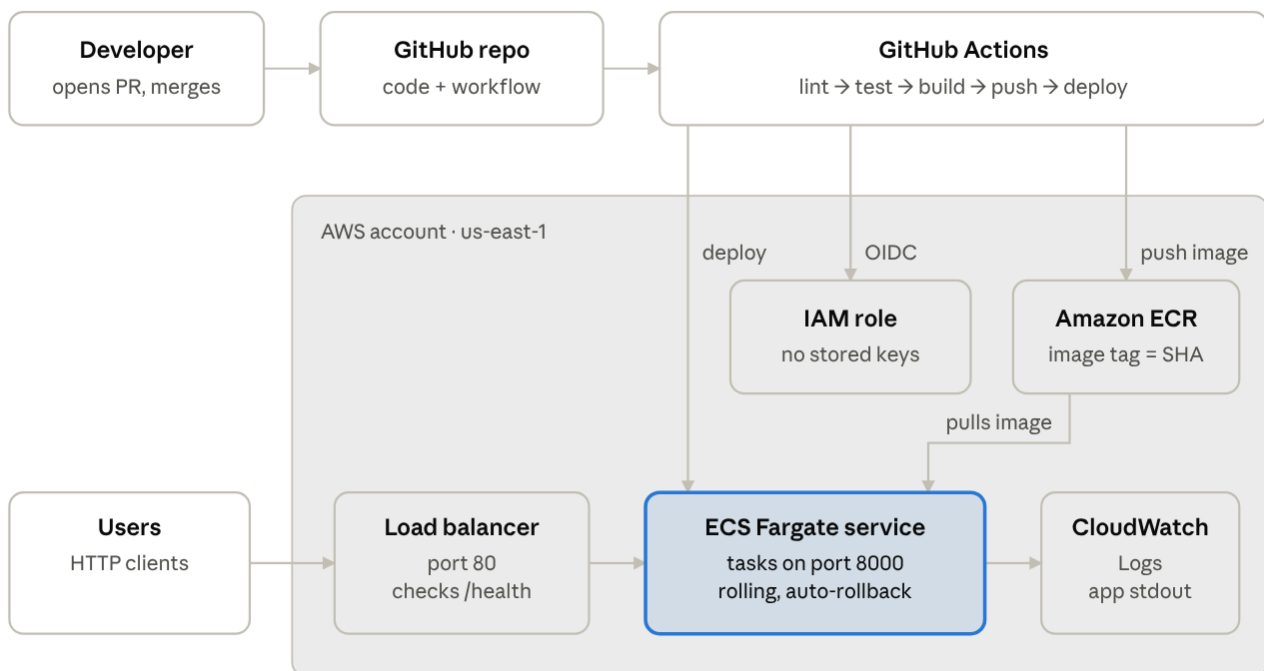
Step 7 — Architecture, deployment runbook and troubleshooting

This section is what goes into your `README.md` as deliverable 7. A merge to `main` reaches production in about 5–8 minutes with no manual step, and a failed release rolls itself back.

7.1 Architecture

The pipeline has two halves: a delivery path (GitHub → ECR → ECS) and a request path (user → ALB → Fargate task). They meet only at the ECS service, which pulls the new image and swaps tasks behind the load balancer.

Every merge to main flows through GitHub Actions into ECS Fargate



CI/CD and runtime architecture · 9 components

The top row is the delivery path; the shaded AWS account holds the runtime path. Actions authenticates through the IAM role, pushes to ECR and tells ECS to deploy; ECS pulls the SHA-tagged image and the load balancer only sends traffic to tasks that pass `/health`.

7.2 Deployment runbook

1. Create a feature branch, commit, and open a pull request to `main`.
2. CI runs **Lint and test** and **Build Docker image** (with a container smoke test). Fix anything red.
3. Merge the PR. The push to `main` triggers **Build and push to ECR**, tagging the image with the commit SHA.
4. **Deploy to ECS Fargate** renders a new task-definition revision with that image and updates the service.
5. ECS starts new tasks, the ALB marks them healthy on `/health`, then drains and stops the old tasks (rolling, zero downtime).
6. The job waits for service stability, then confirms `GET /health` returns the new SHA.
7. If new tasks keep failing health checks, the circuit breaker rolls back to the last good revision automatically.

Manual rollback to a known-good revision: ECS → `task-api-cluster` → `task-api-service` → **Update service** → Revision: pick the last good one (e.g. `task-api:4`) → **Update**. ECS rolls back the same zero-downtime way it deploys.

Where to look day to day:

What you want	Where in the console
Live application logs	ECS → service → Logs tab, or CloudWatch → Log groups → <code>/ecs/task-api</code> → Live tail
Why a deploy is stuck	ECS → service → Events tab (newest first)
Why a task stopped	ECS → service → Tasks tab → filter Stopped → open the task → Stopped reason
Is the load balancer sending traffic?	EC2 → Target Groups → <code>task-api-tg</code> → Targets tab (healthy / unhealthy + reason)
CPU and memory	ECS → service → Health and metrics tab (or CloudWatch → Container Insights)
Scale out	ECS → service → Update service → Desired tasks <code>2</code> → Update

7.3 Troubleshooting

Symptom	Likely cause	Fix
<code>curl localhost:8000</code> fails but container is running	uvicorn bound to 127.0.0.1	Keep <code>--host 0.0.0.0</code> in the Dockerfile <code>CMD</code>
<code>ModuleNotFoundError: app</code> in tests	Running pytest from wrong folder / missing <code>__init__.py</code>	Run from repo root; keep <code>app/__init__.py</code> and <code>tests/__init__.py</code>
GitHub Actions: <code>Could not assume role with OIDC / Not authorized to perform sts:AssumeRoleWithWebIdentity</code>	Missing <code>id-token: write</code> , or trust-policy <code>sub</code> does not match repo/branch/environment	Add the permission to the job; check owner/repo spelling and both <code>sub</code> entries in 5.3
<code>denied: User is not authorized to perform ecr:InitiateLayerUpload</code>	Deploy role policy points at a different repo or region	Fix the <code>Resource ARN</code> in the role's <code>task-api-deploy</code> inline policy

Symptom	Likely cause	Fix
<code>tag invalid: The image tag already exists</code>	ECR tags are IMMUTABLE and a re-run pushed the same SHA	Re-run is safe to skip; or push a new commit
Task stops with <code>CannotPullContainerError</code>	No route to ECR: private subnet without NAT, or <code>assignPublicIp</code> disabled; or execution role missing	Use public subnets with Public IP turned on in the service networking, or add NAT / ECR VPC endpoints; confirm <code>ecsTaskExecutionRole</code> exists
Task stops with <code>exec format error</code>	Image built for ARM (Apple Silicon)	Let the pipeline build the image; it sets <code>platforms: linux/amd64</code>
Targets <code>unhealthy</code> in the target group, tasks cycle	Health-check path/port wrong, or service SG does not allow the ALB	Target group: path <code>/health</code> , port 8000; SVC_SG must allow 8000 from ALB_SG
ALB returns 503	No healthy targets registered yet	Check EC2 → Target Groups → <code>task-api-tg</code> → Targets tab; wait for the grace period; read task logs
ALB returns 504	Security group or app hanging	Verify SG rules; check CloudWatch logs for slow requests
Deploy step hangs at "waiting for service stability"	New tasks failing; circuit breaker still retrying	Read the service's Events tab and the stopped task's Stopped reason ; fix and push
<code>iam:PassRole</code> <code>AccessDenied</code> in deploy job	Deploy role cannot pass the execution role	Check the <code>PassExecutionRole</code> statement ARN
Tasks disappear after deploy / data differs between requests	In-memory storage is per task and resets on each deploy	Expected for this assignment; move to DynamoDB or RDS for persistence

7.4 Security and production hardening (talking points)

- No static AWS keys: OIDC with a role scoped to one repo and branch/environment.
- Non-root container, pinned dependencies, ECR scan on push, immutable image tags.
- Service reachable only through the ALB security group; add HTTPS via ACM and redirect 80 → 443.
- For CDIS production: private subnets + NAT or VPC endpoints, WAF on the ALB, secrets from AWS Secrets Manager or SSM Parameter Store, and a persistent datastore.

7.5 Cost and cleanup

Running cost is roughly the ALB (about \$16–20/month) plus one 0.25 vCPU / 0.5 GB Fargate task (about \$9/month) in `us-east-1`; check current AWS pricing. Delete everything after the demo:

Delete in this order (each step depends on the one before):

1. **ECS** → cluster → `task-api-service` → **Update service** → Desired tasks `0` → Update; then **Delete service** (type `delete` to confirm).
2. **ECS** → Clusters → `task-api-cluster` → **Delete cluster**.
3. **EC2** → **Load Balancers** → `task-api-alb` → Actions → **Delete load balancer**. Wait about a minute.
4. **EC2** → **Target Groups** → `task-api-tg` → Actions → **Delete**.
5. **EC2** → **Security Groups** → delete `task-api-svc-sg`, then `task-api-alb-sg` (if it refuses, wait another minute for the load balancer's network interfaces to disappear).
6. **ECR** → `task-api` → **Delete** (removes all images).
7. **CloudWatch** → **Log groups** → `/ecs/task-api` → Actions → **Delete**.
8. Optional: **IAM** → delete roles `github-actions-task-api` and `ecsTaskExecutionRole`, and the GitHub identity provider, if nothing else uses them.

Demo and Q&A preparation

A 10-minute demo that shows the pipeline working beats any slide. Suggested order: open `/docs` on the live ALB URL → create a task → change the `description` text in `main.py`, open a PR, show CI going green → merge → watch the deploy job → refresh `/health` and show the new SHA.

Likely question	Short answer
Why FastAPI?	Type-hint based validation via Pydantic, async support, and free OpenAPI docs; less code than Flask for the same guarantees.
Why a multi-stage Dockerfile?	Build tools stay out of the runtime image: smaller, faster to pull, smaller attack surface.
Why ECS Fargate over EC2?	No servers to patch or scale; pay per task; native ALB, IAM and CloudWatch integration.
Why ECS over EKS?	One small service does not justify running Kubernetes; ECS is simpler and has no control-plane fee.
How do you avoid storing AWS keys in GitHub?	GitHub OIDC: the job gets a short-lived token and assumes an IAM role limited to this repo's <code>main</code> branch and <code>production</code> environment.
How is a bad release prevented?	Tests gate the build; the image is smoke-tested; ALB health checks gate traffic; the circuit breaker auto-rolls back.
How do you roll back?	Automatic via circuit breaker, or <code>update-service</code> to a previous task-definition revision (runbook 7.2).
Why tag images with the commit SHA, not <code>latest</code> ?	Every running task traces to one commit; immutable tags stop silent overwrites; rollbacks are exact.
What is the difference between the Docker <code>HEALTHCHECK</code> , ECS health check and ALB health check?	Docker's is for local runs; ECS's restarts an unhealthy container; the ALB's decides whether a task receives traffic.
What are the limits of this design?	In-memory data is lost on redeploy and not shared between tasks; HTTP only; public subnets. Next steps: DynamoDB, HTTPS via ACM, private subnets.
How would you add environments (dev/staging/prod)?	One task definition and service per environment, GitHub Environments with approvals, and a promote-by-SHA step between them.

Likely question	Short answer
How would you manage the infrastructure long-term?	Move the CLI steps into Terraform or AWS CDK so the whole stack is versioned and reproducible.

Final submission checklist

- ☐ Repo contains app, tests, Dockerfile, `.dockerignore`, workflow, task definition, README
- ☐ `pytest` passes locally and in Actions (screenshot the green run)
- ☐ Image visible in ECR with a SHA tag and scan results
- ☐ ECS service stable; `http://<ALB_DNS>/health` returns the current SHA
- ☐ README includes the architecture diagram, runbook and troubleshooting table from Step 7
- ☐ Confirmed AWS target, region and account with the CDIS department
- ☐ Resources deleted after grading (7.5) to avoid charges